GUIA PRÁTICO / 01

Como criar endpoints e integrá-los à API

Planejamento Acadêmico - TP Eng2

Guia para a equipe de C++ e React. Baseado nos arquivos locais do projeto em 20/10/2026. C++17, Crow, Firebase Authentication e Firestore.

Objetivo
Acrescentar uma operação à API sem colocar HTTP nas entidades nem duplicar as regras acadêmicas nas telas. O exemplo será consultar uma disciplina pelo código.

O que é um endpoint?

É a combinação de um **método HTTP**, um **caminho** e um **contrato de entrada e saída**. Uma função C++ só fica acessível ao front quando uma rota registrada no Crow chama essa função e transforma o resultado em uma resposta HTTP.

Método	Uso típico	Exemplo
GET	Consultar, sem alterar dados	GET /api/curriculum
POST	Criar um recurso ou executar uma ação	Exemplo futuro: criar disciplina
PUT	Criar/substituir o recurso identificado; pode ser idempotente	PUT /api/history/BCC201
DELETE	Excluir o recurso identificado	DELETE /api/history/BCC201

O que já existe no MVP

Caminho	Operações disponíveis
/api/me	GET: identidade validada pelo token
/api/courses	GET: cursos cadastrados
/api/profile	GET e PUT: consultar e salvar o próprio perfil
/api/curriculum	GET: matriz do curso do aluno
/api/history	GET: conclusões persistidas
/api/history/{codigo}	PUT e DELETE: registrar/remover conclusão
/api/academic/summary	GET: progresso, horas e elegibilidade

A rota pública GET /api/health verifica o processo. As operações acadêmicas exigem Authorization: Bearer <ID token>.

1. Escolha a camada certa

As pastas de backend/include/ e backend/src/ espelham as mesmas responsabilidades. O cabeçalho declara o contrato; o .cpp implementa o comportamento.

Camada	Responsabilidade	Exemplos
domain/	Entidades e regras acadêmicas; sem Crow ou Firebase.	AlunoImpl, CursoImpl, DisciplinaImpl
application/	Orquestrar a operação e usar o repositório.	AcademicService; AcademicRepository
http/	Receber a requisição, validar JSON e retornar JSON.	academic_routes.cpp; AcademicJson.cpp
infrastructure/auth/	Validar o token e obter a identidade.	RequestAuthenticator.cpp
infrastructure/firestore/	Executar leitura e escrita no Firestore.	FirestoreRepository.cpp
config/	Configurar host e porta do processo.	config.cpp

Como a chamada percorre o projeto

React envia o token → a rota autentica a requisição → o serviço executa o caso de uso → o repositório acessa o Firestore → o serviço utiliza o domínio → a rota serializa a resposta.

O UID vem de RequestContext, criado a partir do token validado. Não aceite uid, userId ou um papel de coordenador enviados pelo navegador como prova de identidade ou autorização.

Antes de escrever código, defina o contrato

Decisão	Exemplo didático a implementar
Operação	Consultar uma disciplina da matriz do aluno
Método e caminho	GET /api/subjects/{codigo}
Entrada	Código no caminho, por exemplo BCC201
Autorização	Aluno autenticado; somente sua matriz
Saída	id, name, hours, period, prerequisites, requirementsKnown
Erros principais	401: sem autenticação; 409: perfil incompleto; 404: disciplina ausente

Exemplo novo, não endpoint já entregue
As páginas seguintes mostram alterações a fazer. O guia não adiciona /api/subjects/{codigo} ao repositório automaticamente. O contrato atual está em backend/openapi.json.

2. Implemente serviço e saída JSON

A. Declare o método no serviço

Em backend/include/application/AcademicService.hpp, dentro de public: de AcademicService, acrescente:

```
SubjectRecord subject(const RequestContext& context,
                      const std::string& id);
```

B. Implemente o caso de uso

Em backend/src/application/AcademicService.cpp, dentro de namespace bcc323. O arquivo já inclui <algorithm>. curriculum(context) carrega o curso do perfil autenticado.

```
SubjectRecord AcademicService::subject(
    const RequestContext& context, const std::string& id) {
    const auto catalog = curriculum(context);
    const auto found = std::find_if(
        catalog.subjects.begin(), catalog.subjects.end(),
        [&](const SubjectRecord& s) { return s.id == id; });
    if (found == catalog.subjects.end()) {
        throw AppError(404, "subject_not_found",
                       "Disciplina fora da matriz do seu curso.");
    }
    return *found;
}
```

C. Acrescente a conversão para JSON

Declare em backend/include/http/AcademicJson.hpp e implemente em backend/src/http/AcademicJson.cpp, dentro de namespace bcc323:

```
// Declaracao no .hpp:
crow::json::wvalue toJson(const SubjectRecord& subject);

// Implementacao no .cpp:
crow::json::wvalue toJson(const SubjectRecord& s) {
    crow::json::wvalue out;
    out["id"] = s.id;
    out["name"] = s.name;
    out["hours"] = s.hours;
    out["period"] = s.period;
    out["prerequisites"] = s.prerequisites;
    out["requirementsKnown"] = s.requirementsKnown;
    return out;
}
```

Essa consulta usa o repositório já existente: não precisa criar uma consulta Firestore dentro da rota. O estado individual da disciplina continua disponível em GET /api/academic/summary.

3. Registre a rota no Crow

Em backend/src/http/academic_routes.cpp, coloque o trecho abaixo **dentro de register_academic_routes(crow::SimpleApp& app)**, junto às outras rotas:

```
CROW_ROUTE(app, "/api/subjects/<string>")
([](const crow::request& request, const std::string& id) {
    return handle(request,
        [&](const RequestContext& context,
            AcademicService& service) {
            return toJson(service.subject(context, id));
        });
});
```

O marcador <string> corresponde ao parâmetro id da função. Neste projeto, uma rota declarada sem .methods(...) usa GET. Para escrita, declare o método explicitamente.

Por que usar handle(request, ...)?

O helper já autentica, limita o corpo da requisição, cria o repositório e o serviço, trata AppError e falhas de autenticação e aplica Cache-Control: no-store. Ele está no namespace anônimo deste .cpp: **não pode ser chamado diretamente em outro arquivo**.

Como essa rota entra no executável?

```
// src/main.cpp ja executa:
bcc323::register_routes(app);

// src/http/routes.cpp ja chama:
register_academic_routes(app);
```

Ao adicionar a rota na função já registrada, não altere main.cpp nem crie outro servidor. A próxima compilação incorpora a operação.

Se precisar criar outro módulo de rotas

Crie um .hpp em include/http/ e um .cpp em src/http/, declare a função register_... e chame-a a partir de register_routes. Antes, extraia o wrapper de autenticação/erros para uma função HTTP compartilhada; não suponha que handle está exportado.

Atenção ao CMake

Os .cpp em src/http/, src/domain/ e src/infrastructure/ são coletados pelos globs atuais. AcademicService.cpp está listado explicitamente. Se criar outro .cpp de aplicação, adicione-o ao alvo, por exemplo:

```
target_sources(academic-application PRIVATE
    src/application/SubjectService.cpp
)
```

Use esse trecho somente se o arquivo existir. Se apenas editar AcademicService.cpp, não é necessário alterar o CMake.

4. Operações que alteram dados

Uma rota de escrita precisa validar entrada, aplicar regras, verificar autorização e confirmar persistência. Retornar sucesso sem salvar não conclui a operação.

Modelo já existente: salvar o próprio perfil

```
// Dentro do handler de /api/profile:
const auto input = crow::json::load(request.body);
const auto profile = profileFromJson(input);
const auto saved = service.saveProfile(context, profile);
crow::json::wvalue body;
body["profile"] = toJson(saved);
return body;
```

Esse trecho reutiliza o parser atual. Ele valida os campos aceitos e o período inteiro de 1 a 20. O serviço resolve o curso cadastrado. O repositório grava usando o UID validado e usa timestamp do servidor ao criar o perfil.

Etapa	O que conferir
Entrada HTTP	JSON válido; tipos; campos obrigatórios; limites; campos extras; IDs.
Serviço	Operação permitida; existência do recurso; regras das classes do domínio.
Repositório	Caminho correto no Firestore; gravação confirmada; falhas propagadas.
Resposta	Status coerente; JSON estável; mensagem que o front consegue tratar.

Como evoluir a frente 2 da divisão de tarefas

Tarefa	Contratos possíveis - ainda a implementar
Cadastrar/editar disciplinas	POST /api/courses/{curso}/subjects; PUT /api/courses/{curso}/subjects/{codigo}
Vincular pré-requisitos	PUT /api/courses/{curso}/subjects/{codigo}/prerequisites
Turmas e horários	POST /api/courses/{curso}/offerings; PUT /api/courses/{curso}/offerings/{turma}

Pré-condição: autorização administrativa
O MVP atual não define autorização de coordenador para editar o catálogo. Antes de expor essas escritas, definam uma fonte de papel confiável, a verificação no servidor e as permissões de persistência. Login sozinho não concede esse poder. Não basta liberar a rota ou confiar em role enviado pelo front.

Ao incluir uma operação no repositório, atualize a interface AcademicRepository, a implementação FirestoreRepository e os repositórios falsos dos testes. Para requisitos, reutilize DisciplinaImpl::addPreRequisito, inclusive a prevenção de ciclos.

5. Consuma e teste a operação

Acrescente o método no cliente do front

Em frontend/src/lib/academic.ts, importe o tipo Subject de ./api e acrescente ao objeto academicApi:

```
subject: (id: string) =>
  api<Subject>(`/subjects/${encodeURIComponent(id)}`),
```

api() já acrescenta /api, obtém o token Firebase, envia Authorization e trata respostas de erro. A tela chama o cliente e mantém seus estados de carregamento, erro e resultado. Não coloque localhost:8081 nos componentes.

```
try {
  const subject = await academicApi.subject("BCC201");
  // Atualize o estado da tela com o resultado.
} catch (error) {
  // Mostre a mensagem e ofereca nova tentativa.
}
```

Prepare o ambiente local

```
# Terminais separados, a partir da raiz do projeto:
./scripts/dev-local.ps1 firebase
node scripts/seed-demo.mjs
./scripts/dev-local.ps1 backend
./scripts/dev-local.ps1 frontend
```

Execute o seed após os emuladores estarem ativos. Ele cadastra somente um recorte ilustrativo em projeto demo. Cadastre/entre com um aluno e complete o perfil. No Linux, use bash scripts/dev-local.sh para iniciar os serviços.

Teste manual depois de implementar o exemplo

```
# Substitua pelo ID token de uma sessao de teste.
$token = '<ID_TOKEN>'
curl.exe 'http://127.0.0.1:8081/api/subjects/BCC201' `
  -H "Authorization: Bearer $token"
```

Também teste sem Authorization, com código inexistente e sem perfil completo. O esperado é 401, 404 e 409, respectivamente. Não coloque tokens reais no Git, na documentação nem em capturas compartilhadas.

Adicionar ao código não significa publicar

Os scripts rodam a API localmente. O proxy do Vite existe em desenvolvimento/preview, mas não acompanha o build estático. Para usar a nova rota no site publicado, será necessário publicar o C++ e configurar o encaminhamento remoto de /api, que ainda está pendente no projeto.

6. Testes e entrega para a equipe

Teste o comportamento, não apenas o HTTP 200

Em frontend/e2e/academic-api.spec.ts, dentro do teste que já prepara o aluno, o perfil e headers first, acrescente após implementar o novo endpoint:

```
const response = await request.get('/api/subjects/BCC201', {
  headers: first,
});
expect(response.status()).toBe(200);
expect(await response.json()).toMatchObject({ id: 'BCC201' });

const missing = await request.get('/api/subjects/INVALID', {
  headers: first,
});
expect(missing.status()).toBe(404);
expect((await request.get('/api/subjects/BCC201')).status())
  .toBe(401);
```

Para regras novas, acrescente testes em backend/test/application/. Para gravações, verifique persistência após nova consulta, isolamento entre usuários, falha de validação e comportamento em repetição da operação.

Compilar e verificar no Windows

```
. ./scripts/local-env.ps1
cmake -S backend -B backend/build/native -DBUILD_TESTING=ON
cmake --build backend/build/native --parallel 2
ctest --test-dir backend/build/native --output-on-failure
# Com os servicos ativos, dentro de frontend/:
bun run typecheck
bun run lint
bun run test:e2e
```

Encerre o backend antes de recompilar no Windows: o executável aberto pode impedir a linkedição. Depois, reinicie-o. Testes HTTP em Python só são habilitados no CTest quando o interpretador está disponível.

Checklist do PR

Verificação	Pronto quando...
Contrato	Método, caminho, entrada, saída, erros e autorização estão definidos em backend/openapi.json.
Implementação	A rota foi registrada; serviço usa o domínio; persistência fica no repositório.
Integração	Cliente academic.ts atualizado; testes de sucesso e falha passam.
Equipe	Responsável da frente 2 ou 3 consegue usar o contrato sem alterar a arquitetura base.

Erros comuns: 404 por esquecer o registro; 405 por método incorreto; 401 por token ausente; referência indefinida por esquecer um .cpp no CMake; dados não persistidos por só alterar objetos em memória.

Referências do próprio projeto: backend/README.md; backend/CONTRATO_MVP.md; backend/openapi.json; frontend/src/lib/academic.ts. Os exemplos deste guia não foram aplicados automaticamente ao código.